Project Assignment 5: Connecting the Tables

Form Content

The Scenario

In Assignment 4, you analyzed platform performance and identified producers with unusually high failure rates. However, your results only included producer IDs. Leadership now wants to know who these producers are and what might explain their performance.

In this assignment, you will use SQL joins to connect event data with information about producers, actors, and catalog records. You will also investigate an important question: **Are there producers who have never had an event?** These producers did not appear in your previous analysis because they have no matching records in the event table. By using outer joins, you can identify them and investigate why they may not be active. Finally, you will explore how SQL joins affect database performance. You will examine an execution plan to understand how the database processes your queries and what might happen as the platform grows.

Your goal is to create a more complete picture of platform activity and recommend actions based on your findings.

Your database

Keep using the ex603_data database you loaded in Unit 3 — the same dataset, the same tables. If you need to set it up again, the full instructions are in Assignment 3. Nothing new to install this unit.

Overview

In this assignment, you will practice combining data from multiple tables using SQL joins. You will learn how to:

- Use INNER JOIN to combine related records from multiple tables.
- Join three tables to create a more informative report.
- Work with junction tables to explore many-to-many relationships.
- Use self joins to compare records within the same table.
- Use LEFT JOIN, RIGHT JOIN, and FULL OUTER JOIN to identify matching and missing records.
- Find unmatched records using NOT EXISTS and EXCEPT.

- Understand Cartesian products using CROSS JOIN.
- Use EXPLAIN to examine how PostgreSQL executes a query.

You will apply these skills to create a report and recommendations for the platform's leadership team.

Where to write your SQL
One file per task in /queries/unit5/ — q5_1.sql through q5_3.sql. Head every query with a comment saying which requirement it answers and what question it asks in plain language.

Task 5.1: Connect Producers to Their Data

Your first task is to connect information from different tables so leadership can better understand the producers identified in Assignment 4.

The questions

1. Who are the people behind the events? Write a query that joins three tables to display the actor who performed each event, the producer associated with the event, and the event's date and time.
2. Do the producers identified in Assignment 4 share certain catalog credentials or tags? Write a query that connects the producer table to the catalog table through the junction table. Return the producers associated with your theme's catalog tag.
3. How do the producers compare with one another? Write a self join to compare records within the producer table. Use the results to explore whether the producers identified in Assignment 4 share characteristics with other producers or whether differences in activity levels may explain their performance.

Write up
In /analysis/unit5.md, briefly answer the following questions:

1. What feature or report on your platform could use the three-table join from Question 1? What information would it display?
2. Why is the junction table important for answering Question 2? What would make this analysis more difficult without it?

Task 5.2: Find Records That Joins May Exclude

Part A: Compare INNER JOIN and LEFT JOIN
Write two queries to compare the results of an inner join and a left join.

1. Use LEFT JOIN to return all producers, including those who have never had an event.
2. Write an equivalent query using INNER JOIN.
3. Compare the results and determine how many producers are excluded by the inner join.
4. Explain which producers are missing and why.

Your results must include a count of the producers excluded by the inner join.

Hint: When counting producers, use their unique identifiers so that producers with multiple events are not counted more than once.

Part B: Explore RIGHT JOIN and FULL OUTER JOIN

Next, examine the same relationship using two additional join types.

1. Write a RIGHT JOIN that returns the same records as your LEFT JOIN from Part A.
2. Compare the results to confirm that they match.
3. Write a FULL OUTER JOIN that includes matching and unmatched records from both tables.
4. Explain what a full outer join can reveal that a left join or right join alone may not show.

Be sure to verify your results rather than assuming the queries are equivalent.

Part C: Find Producers with No Events Using Two Approaches

Write two queries that identify producers who have never had an event and compare them.

✓ NOT EXISTS.

✓ EXCEPT.

Run both queries and compare their results.

In one or two sentences, explain how the approaches differ and confirm whether they identify the same producers.

Write up

In /analysis/unit5.md, answer the following questions:

1. Which producers were excluded by the inner join? Report their identifiers.
2. What does it mean when a producer has no events? What are some possible explanations?
3. How could a report based only on an inner join give leadership an incomplete picture of the platform?
4. Why might leadership want to identify producers who have never been active?
5. What action could leadership take based on these findings?

Distinguish between what your data shows and what would require further investigation. For example, a producer with no events may be inactive, but the data alone may not explain why.

Task 5.3: Understand Join Performance and Execution Plans

In this task, you will explore how joins affect query performance and how PostgreSQL decides to execute a query.

Part A: Explore CROSS JOIN

Write a CROSS JOIN between two small tables.

Then:

1. Count the rows in each source table.
2. Run the CROSS JOIN and count the resulting rows.
3. Show how the result matches the multiplication of the two source table counts.
4. Explain when a Cartesian product might be useful and when it could indicate a query error.

For example, if one table has 5 rows and another has 4 rows, the cross join produces 20 rows.
Choose small tables so the result is manageable.

Part B: Examine a Query Execution Plan

Return to your three-table join from Task 5.1, Part A. Use PostgreSQL's EXPLAIN command to examine how the database plans to execute the query.

```
EXPLAIN
SELECT ...    -- your three-table join from 5.1
;
-- EXPLAIN ANALYZE also runs the query and reports actual timings.
-- Safe here; be careful with it on statements that modify data.
```

Review the execution plan and complete the following:

1. Capture the full execution plan.
2. Identify the join algorithm or algorithms used, such as Nested Loop, Hash Join, or Merge Join.
3. Explain why PostgreSQL may have selected these algorithms, considering table sizes and available indexes.
4. Predict which part of the query might become less efficient as the dataset grows.

You may also use EXPLAIN ANALYZE to measure actual execution behavior. Unlike EXPLAIN, it runs the query. It is appropriate for your read-only SELECT query, but use caution with statements that modify data.

Write up

In /analysis/unit5.md, answer the following questions:

1. Give an example of when a Cartesian product could be useful on your platform, or explain why you would not need one.
2. Which join algorithms did PostgreSQL choose for your query?
3. What information in the execution plan helps explain those choices?
4. Why is it useful to understand the selected join algorithms?
5. What changes to the data or database might make another join algorithm more appropriate?

Support your explanation with observations from your execution plan.

Task 5.4: Write Your Recommendations

The memo

In /analysis/unit5-finding.md.

You now know which producers fail often and which never started. Recommend a concrete action for each group. The producers who never started demand a different response from the ones who fail often, and saying so precisely is the point. Write for a non-technical reader.

Reflection (150–250 words)

In Task 5.3, you examined the execution plan for your three-table join. Reflect on the following question: **What changes to your data or database schema might cause PostgreSQL to choose a different join algorithm?**

Consider factors such as table size, indexes, and the number of matching records. Explain how these changes could influence query performance. Include your reflection in /analysis/unit5.md.

Deliverable: Your Repository

Submit the URL of your public GitHub repository to Question 1. Nothing is attached to the submission.

Path	What goes there
/queries/unit5/q5_1.sql	Task 5.1 — the three-table join, the junction traversal, and the self join.
/queries/unit5/q5_2.sql	Task 5.2 — LEFT, RIGHT, and FULL OUTER joins, plus NOT EXISTS and EXCEPT.
/queries/unit5/q5_3.sql	Task 5.3 — the CROSS JOIN and the EXPLAIN statement.
/analysis/unit5.md	The write-ups for Tasks 5.1, 5.2, and 5.3, plus the reflection and your reading of the plan.
/analysis/unit5-finding.md	Task 5.4 — the recommendation memo.
/screenshots/	The screenshots below.

Screenshots

A screenshot counts only if it shows the work: a full, uncropped capture in which the query you ran and the output it produced are both legible in the same image.

✓ Each query running with its result set visible.
✓ The EXPLAIN output, showing the command typed and the full plan returned in one capture.
✓ The CROSS JOIN row count, with the source table counts visible so the multiplication is checkable.

Before you submit

✓ The repository is public — verify by opening the URL in a logged-out browser window.
✓ Every .sql file runs against the provided dataset without editing.

Question 1

Submit your repository URL.

Use the editor to format your answer

Questions Filter (1) ▼

Save and Close

Submit